

Arquitectura de Software

Gastos Compartidos API — cómo está construida y cómo se usa

Este documento explica, sin dar nada por sabido, cómo está organizada por dentro esta aplicación y de qué forma se conecta con cualquier interfaz — una web, una app de móvil o cualquier otro programa. Cada palabra técnica se define la primera vez que aparece, y al final hay un glosario completo.

1 La idea central: dos mitades separadas

Toda aplicación moderna se divide en dos partes que viven separadas y se comunican entre sí:

FRONTEND

La cara visible: botones, pantallas, lo que el usuario toca

BACKEND

El cerebro oculto: guarda datos, hace cálculos, aplica las reglas

Lo que hemos construido es el backend — el cerebro. No tiene pantallas ni botones: es un programa que se queda esperando órdenes, las procesa y devuelve datos. La parte visible (frontend) es una pieza aparte que se conecta a él. Un mismo backend puede servir a la vez a una web, a una app de móvil y a un bot — todos le piden datos al mismo cerebro.

API (Application Programming Interface)

El conjunto de «enchufes» por los que otro programa se conecta a nuestro backend para pedirle cosas. No es una web ni una app: es la lista de operaciones disponibles (registrar usuario, crear gasto, ver saldos...) y el formato en que se piden y se devuelven. Nuestra API es de tipo **REST**, el estilo más extendido, que usa direcciones web (URLs) y verbos de internet.

2 La arquitectura en capas

El backend no es un bloque único: está dividido en **capas**, cada una con una sola responsabilidad. Una petición las atraviesa de arriba abajo. Separarlas así hace que el código sea fácil de entender, probar y cambiar sin romper el resto.

FastAPI — Routers

Reciben la petición de internet y deciden qué función la atiende



Pydantic — Schemas

Validan que los datos que llegan tengan el formato correcto



Services — Lógica de negocio

Los cálculos puros: dividir gastos, simplificar deudas



SQLAlchemy — Models

Traducen entre objetos de Python y filas de la base de datos



PostgreSQL — Base de datos

Guarda los datos de forma permanente en el disco

Cada tecnología de esas capas es simplemente **código gratuito y de código abierto** que se descarga y se importa en nuestro programa. No son empresas ni servicios externos a los que haya que registrarse: son herramientas que corren dentro de nuestra propia máquina.

3 El ciclo de una petición, paso a paso

Veamos qué ocurre exactamente cuando alguien crea un gasto desde su móvil. La app envía un mensaje al backend y recibe una respuesta; por el camino se atraviesan todas las capas:

1	La app envía la petición POST /groups/123/expenses con datos en formato JSON: {"description":"Cena","amount":30}
2	Pydantic valida los datos ¿El importe es un número positivo? ¿Están los campos obligatorios? Si no, rechaza con error automático.
3	Se comprueba la identidad Se lee el token de seguridad (JWT) para saber quién hace la petición y si tiene permiso.
4	La lógica de negocio calcula El servicio de reparto divide el gasto entre los miembros según el método elegido.
5	Se guarda en la base de datos SQLAlchemy convierte el resultado en filas y PostgreSQL las escribe en disco.
6	Se devuelve la respuesta El backend responde con JSON y un código: 201 = creado con éxito.

JSON (JavaScript Object Notation)

El formato de texto en que viajan los datos entre la app y el backend. Es una lista de «campo: valor» legible por humanos y por máquinas, p.ej. {"amount": 30, "description": "Cena"}. Es el idioma común: da igual que el frontend esté hecho en un lenguaje y el backend en otro, ambos entienden JSON.

Endpoint

Cada combinación concreta de **verbo + dirección** a la que se puede llamar. POST /groups (crear un grupo) es un endpoint; GET /groups (listar grupos) es otro distinto, aunque compartan dirección. Los verbos habituales son GET (leer), POST (crear), PATCH (editar) y DELETE (borrar).

JWT (JSON Web Token) — el token de seguridad

Un código cifrado que el backend entrega al hacer login y que la app adjunta en cada petición posterior para demostrar quién es, sin reenviar la contraseña. Es como una pulsera de festival: te la dan al entrar y basta enseñarla para acceder después.

4 El stack: qué hace cada tecnología

Estas son las herramientas que componen el proyecto. La columna «analogía» ayuda a fijar la idea.

Pieza	Qué es	Para qué sirve aquí	Analogía
Python	Lenguaje de programación	El idioma en que está escrito todo el backend	El idioma base
FastAPI	Librería de Python para APIs	Convierte funciones en endpoints accesibles por internet	El recepcionista
Pydantic	Librería de validación	Comprueba que los datos que entran son correctos	El portero/filtro
SQLAlchemy	ORM (traductor a base de datos)	Convierte objetos Python en filas de tablas y viceversa	El traductor
PostgreSQL	Base de datos	Guarda los datos de forma permanente y fiable	El archivador
Alembic	Gestor de migraciones	Versiona los cambios en la estructura de las tablas	El control de versiones del esquema
Docker	Empaquetado en contenedores	Mete la app y sus dependencias en cajas que corren igual en cualquier sitio	El contenedor de barco

ORM (Object-Relational Mapping)

Una capa que traduce automáticamente entre los **objetos** del código (Python) y las **filas** de una base de datos relacional (SQL). Gracias a él escribimos `db.add(gasto)` en vez de teclear a mano las órdenes `SQL INSERT INTO....` SQLAlchemy es el ORM que usamos.

Docker y «contenedor»

Un contenedor es una caja aislada que lleva dentro tu app **junto con todo lo que necesita** (Python, librerías, configuración). Como lleva todo dentro, funciona igual en tu Mac, en otro ordenador o en un servidor en la nube. Docker Desktop es el motor que enciende esas cajas; debe estar abierto para que la app corra localmente.

Migración (Alembic)

Un archivo versionado que describe un cambio en la estructura de la base de datos (crear una tabla, añadir una columna...). En lugar de modificar las tablas a mano, se aplican migraciones en orden, de modo que cualquier copia del proyecto llega exactamente al mismo esquema.

5 Cómo se conecta a distintas plataformas

Aquí está la gran ventaja de separar backend y frontend: **el mismo backend sirve a cualquier tipo de interfaz**. Todas hablan con él igual — enviando peticiones y recibiendo JSON. No hay que reescribir nada del backend para añadir una nueva plataforma.



Qué tendrías que construir en cada caso

Plataforma	Qué añades encima del backend	Tecnología típica
Aplicación web	Una interfaz que corre en el navegador y llama a la API	React, Vue, Angular, o HTML+JavaScript
App móvil nativa	Una app instalable que llama a la API por internet	Kotlin/Java (Android), Swift (iOS)
App móvil multiplataforma	Una sola app para Android e iOS a la vez	Flutter, React Native
Bot de chat	Un programa que recibe mensajes y llama a la API	Bibliotecas de Telegram / Discord
Integración / automatismo	Otro sistema que consume la API sin interfaz humana	Cualquier lenguaje con peticiones HTTP

En los cinco casos, el trabajo pesado — validar datos, calcular el reparto, simplificar deudas, guardar en la base de datos, controlar permisos — **ya está hecho en el backend**. El frontend solo se ocupa de mostrar información y recoger lo que el usuario escribe. Por eso un backend bien hecho es una inversión que se reutiliza en todas las plataformas.

6 Dónde vive el backend

Para que las apps puedan conectarse, el backend tiene que estar **encendido y accesible**. Hay tres escenarios según para qué lo quieras:

Escenario	Dónde corre	Coste aprox.
Probar / desarrollar	En tu Mac, con Docker Desktop abierto (localhost:8000)	0 €
Uso real con amigos	En un servidor en la nube encendido 24/7 (Railway, Render, Fly.io)	0–5 €/mes
Producto público	Servidor escalable + dominio propio	Variable

Mientras usas `localhost:8000` en tu ordenador, la dirección `localhost` significa «esta misma máquina»: solo tú puedes verlo. Para que otras personas accedan desde sus móviles, el backend debe estar desplegado en un servidor con una dirección pública de internet.

7 Glosario rápido

Todos los términos técnicos del documento, reunidos para consulta.

Backend La parte oculta de una aplicación: procesa datos, aplica reglas y habla con la base de datos. No tiene interfaz visible.

Frontend La parte visible con la que interactúa el usuario: pantallas, botones, formularios. Vive separada del backend.

API REST El conjunto de operaciones que el backend ofrece al exterior, organizadas como direcciones (URLs) y verbos de internet.

Endpoint Una operación concreta de la API: un verbo (GET/POST/...) más una dirección. P.ej. POST /groups.

HTTP El protocolo (conjunto de reglas) con que los programas se piden y envían información por internet.

Petición / Respuesta El mensaje que un programa envía (petición) y lo que el backend devuelve (respuesta), normalmente en JSON.

JSON Formato de texto para intercambiar datos, en pares «campo: valor». Idioma común entre frontend y backend.

JWT Token de seguridad cifrado que identifica al usuario en cada petición tras el login, sin reenviar la contraseña.

Framework / Librería Código ya escrito por otros que importas en tu programa para no partir de cero. FastAPI y Pydantic lo son.

ORM Traductor automático entre objetos del código y filas de la base de datos. Aquí: SQLAlchemy.

Base de datos Programa que almacena los datos de forma permanente y organizada. Aquí: PostgreSQL.

Migración Archivo versionado que aplica un cambio en la estructura de la base de datos de forma controlada. Aquí: Alembic.

Docker / Contenedor Sistema que empaqueta la app con todo lo que necesita en una caja aislada que corre igual en cualquier máquina.

Servidor Ordenador (normalmente en la nube) encendido de forma permanente para que la app esté siempre accesible.

localhost Nombre que significa «esta misma máquina». Lo que corre en localhost solo lo ves tú.

Swagger UI Página de documentación interactiva que FastAPI genera sola a partir del código, en /docs.

Gastos Compartidos API • Documento de arquitectura • generado para uso personal y de portfolio